



编译原理

引论

丁志军

dingzj@tongji.edu.cn

算术表达式的求解

$$3 + 4 \times 2 + 6$$

- A机器仅具备二目四则运算（ $+$ 、 $-$ 、 $\times$ 、 $\div$ ）的能力

例如： $3+4$

4×2

$2+6$

$8\div 4$

- 现要开发基于A机器的计算器，可以计算形式如下的表达式：

$3 + 4 \times 2 + 6$

- 要把上述表达式拆分成如下几步：

Step1: $T_1 := 4 \times 2$

Step2: $T_2 := 3 + T_1$

Step3: $T_3 := T_2 + 6$

Step4: 输出 T_3



?

- A机器仅具备二目四则运算（ $+$ 、 $-$ 、 $\times$ 、 $\div$ ）的能力
- 现要开发基于A机器的计算器

- 识别操作数、运算符
- 根据表达式的运算规则分析表达式
- 分解表达式为A机器可接受的一组二目运算表达式
- 在A机器上执行，并输出结果

计算器

编译技术

A机器

● 2020年图灵奖获奖人:

- 哥伦比亚大学计算机科学名誉教授**阿尔弗雷德·艾侯(Alfred Aho)**
- 斯坦福大学计算机科学名誉教授**杰弗里·乌尔曼(Jeffrey Ullman)**



■ 获奖理由

◆ 表彰两位在编程语言实现领域对基础算法和理论的贡献。

➤ 编程语言理论和实现

➤ 算法设计和分析

◆ 表彰两位将相关成果系统成书，并通过他们的若干著作影响了几代计算机科学家。

正是编译器的存在，才让人类语言转换为计算机可以理解的 0 和 1。

可以说，是Aho和Ullman的编译技术才使得计算机能看懂人类的编程语言。

全国大学生计算机系统能力大赛

Computer System Development Capability Competition

[CPU设计赛](#)[编译系统赛](#)[操作系统赛](#)[数据库系统赛](#)[智能计算创新赛](#)[智能系统创新赛](#)

2026年全国大学生计算机系统能力大赛

第七届 | 编译系统设计赛

「编译系统挑战赛」

2026年全国大学生计算机系统能力大赛-编译系统设计赛-编译系统挑战赛

即将开启

2025年全国大学生计算机系统能力大赛

第六届 | 编译系统设计赛

编译系统实现赛 · 江苏省

2025年全国大学生计算机系统能力大赛-编译系统设计赛-编译系统实现赛（江苏省）

比赛已结束

2025/04/27--2025/08/05

参赛队伍数量：11

2025年全国大学生计算机系统能力大赛

第六届 | 编译系统设计赛

「编译系统实现赛」

2025年全国大学生计算机系统能力大赛-编译系统设计赛-编译系统实现赛

比赛已结束

2025/08/18--2025/08/22

参赛队伍数量：151

<https://compiler.educg.net/#/>

三、赛道设置

3.1 大赛分编译系统实现和编译系统挑战两个赛道。

编译系统实现赛道面向高等院校在校本科生，分 ARM 和 RIS

编译系统挑战赛道面向高等院校及其他科研机构在校本科生和

3.2 编译系统实现和编译系统挑战赛道均分为初赛和决赛两个阶段
决赛。

3.3 为进一步扩大大赛影响力，推动区域性编译课程的教学和编译
动江苏省省赛试点，省赛章程和技术方案另行发布。

3.4 编译系统挑战赛道由高校和华为公司共同命题，注重考察作品

3.5 大赛对编译系统实现赛道和编译系统挑战赛道分别进行评奖。

和 RISC-V 两个后端分别评奖，编译系统挑战赛道中的所有赛
打分和评奖。

2025 年全国大学生计算机系统能力大赛

编译系统设计赛（华为毕昇杯）

编译系统实现赛道技术方案

详情访问大赛技术平台：<https://compiler.educg.net>

一、评价方式的基本说明

第 1 条 大赛要求各参赛队综合运用各种知识（包括但不限于编译技术、操作系统、计算机体系结构等），构思并实现一个综合性的编译系统，以展示面向特定目标平台的编译器构造与编译优化的能力。

第 2 条 大赛鼓励各参赛队充分了解目标语言及目标硬件平台（CPU 指令集、Cache、各类并行加速能力等）特点，使编译出的目标码能够尽可能利用目标硬件平台能力以提高目标码的运行效率。

第 3 条 为展示参赛队的设计水平，增加竞赛的对抗性，进入决赛的参赛队还需针对目标语言或目标平台的变化，现场调整编译系统。

第 4 条 除本技术方案特别要求、规定和禁止事项外，各参赛队可自行决定编译器体系结构、前端与后端设计、代码优化等细节。

二、初赛评分标准

第 5 条 比赛内容。开发支持特定语言、面向 ARM 硬件平台或 RISC-V 硬件平台的综合性编译系统。

https://compiler.educg.net/?op=4#/index?TYPE=26COM_C

2026年全国大学生计算机系统能力大赛-编译系统设计赛-编译系统挑战赛

编译系统挑战赛



编译系统挑战赛

大赛时刻

章程与技术方案

技术支持

- 2026年全国大学生计算机系统能力大赛编译系统设计赛-编译系统挑战赛-基于AscendNPU_IR的NPU自动融合代码生成与调优-技术方案
- 2026年全国大学生计算机系统能力大赛编译系统设计赛-编译系统挑战赛-基于Triton-Ascend构建昇腾亲和算子-技术方案
- 2026年全国大学生计算机系统能力大赛编译系统设计赛-编译系统挑战赛-动态算子图编译与并行调度-技术方案
- 2026年全国大学生计算机系统能力大赛编译系统设计赛-编译系统挑战赛-仓颉高性能Range_Analysis分析-技术方案
- 2026年全国大学生计算机系统能力大赛编译系统设计赛-编译系统挑战赛-仓颉数据竞争静态检测-技术方案
- 2026年全国大学生计算机系统能力大赛编译系统设计赛-编译系统挑战赛-仓颉代码片段语义检查-技术方案
- 2026年全国大学生计算机系统能力大赛编译系统设计赛-编译系统挑战赛-Java虚拟线程实现机制优化-技术方案
- 2026年全国大学生计算机系统能力大赛编译系统设计赛-编译系统挑战赛-基于进化算法的Triton自动优化系统-技术方案

内容线索

1. 什么叫编译程序

2. 编译过程概述

3. 编译程序的结构

4. 编译程序的生成

5. 总结

程序语言技术的发展

表示机器实际操作的
数字代码

机器语言

C7 06 0000 0002

表示在IBM PC 上
使用的Intel 8x86
处理器将数字2移
至地址0 0 0 0
(16进制) 的指令

以符号形式给出指令
和存储地址

汇编语言

MOV X, 2

类似于数学定义或自
然语言的简洁形式编
写程序的操作

高级语言

X=2

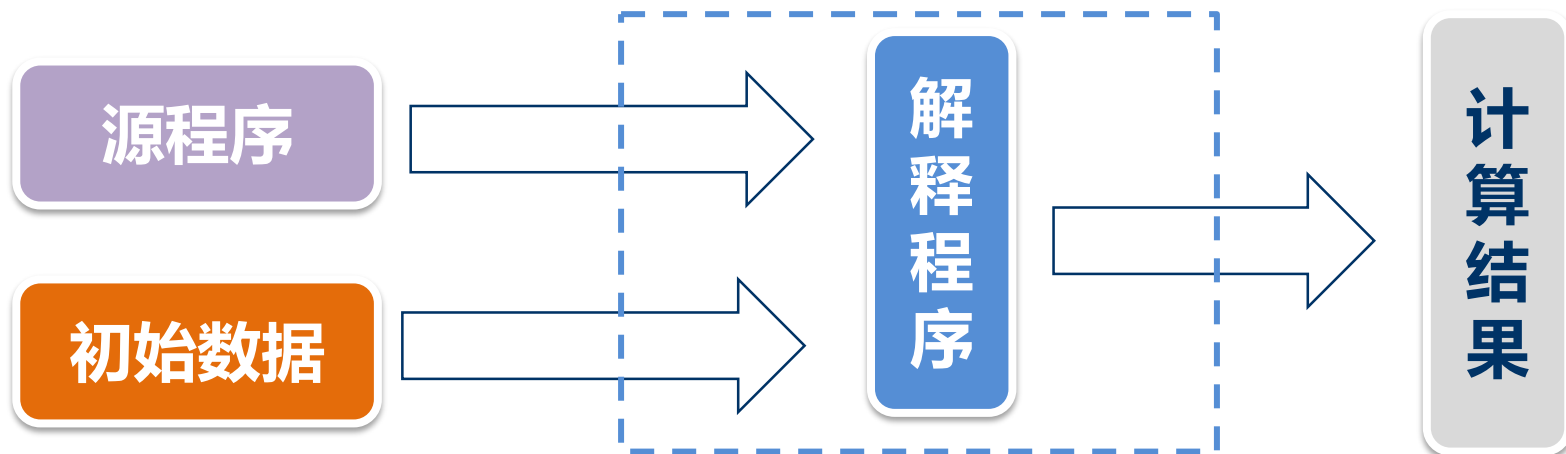
?

程序的执行方式

- 高级语言程序通常采用**解释方式**和**编译方式**两种方式执行

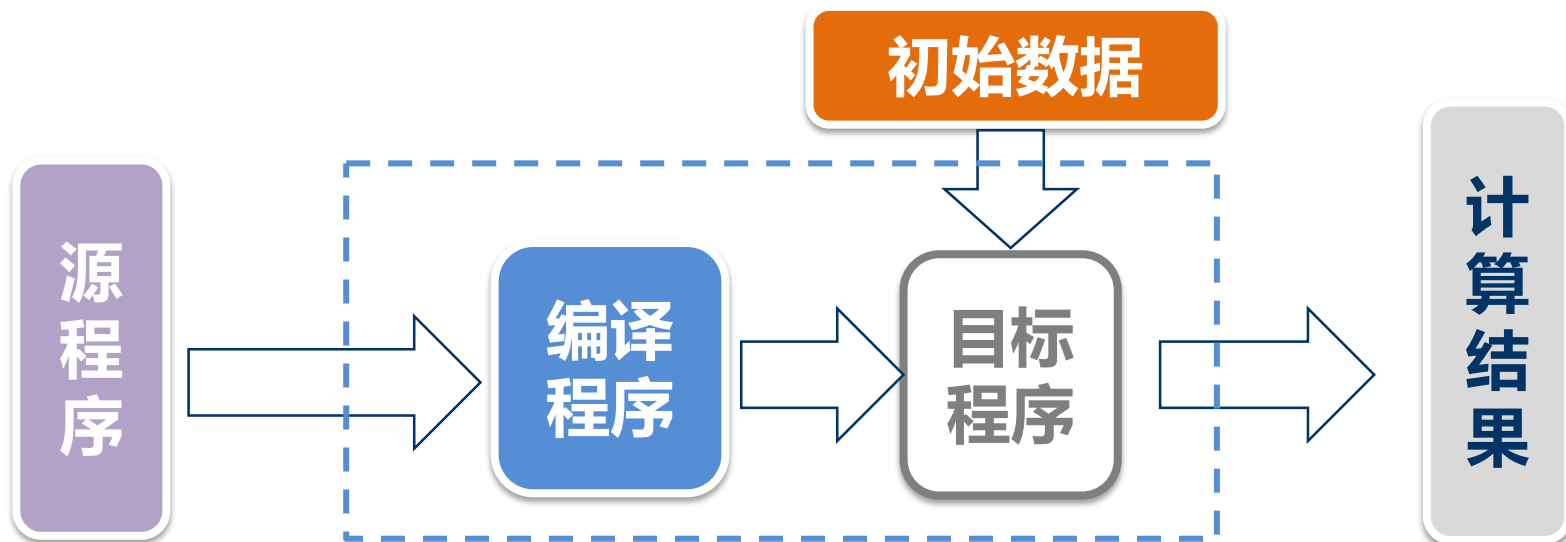
解释程序

边解释边执行源语言程序，不产生目标语言程序。



编译程序

能够把某种语言的程序转换成另一种语言的程序，而后者与前者在逻辑上是等价的。



解释程序和编译程序的区别

	功能	工作结果	实现技术上
编译程序	源程序的一个 转换 系统	源程序的 目标代码	把中间代码转换成目标程序
解释程序	源程序的一个 执行 系统	源程序的 执行结果	执行中间代码

简单的说： 编译就是全文翻译，全部翻译完才执行。
解释就相当于同声翻译，边翻译边执行。

解释程序和编译程序的区别

解释方式

逐个语句地分析和执行

- 如Basic, Ruby、Python
- **优点：**易于查错 启动速度快
内存使用少
跨平台 移植性好
- **缺点：**效率低，运行速度慢

编译方式

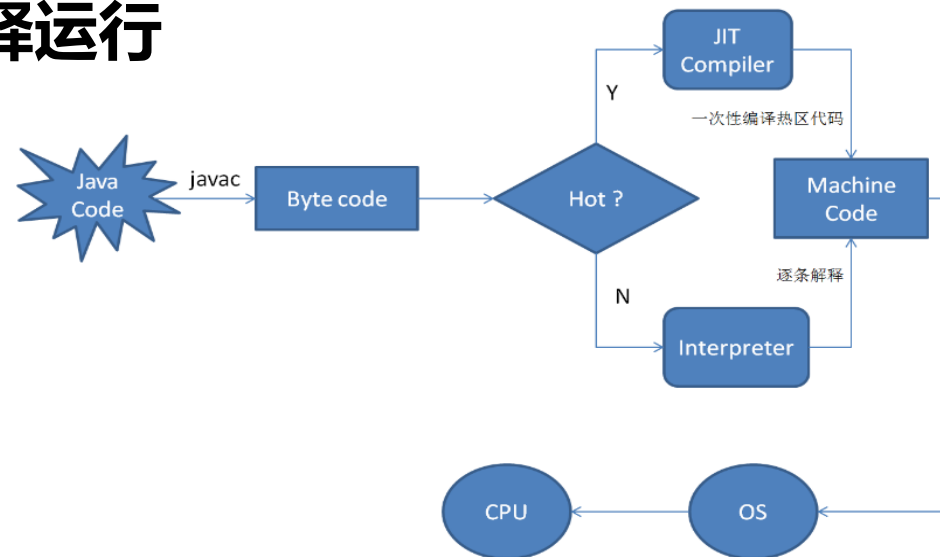
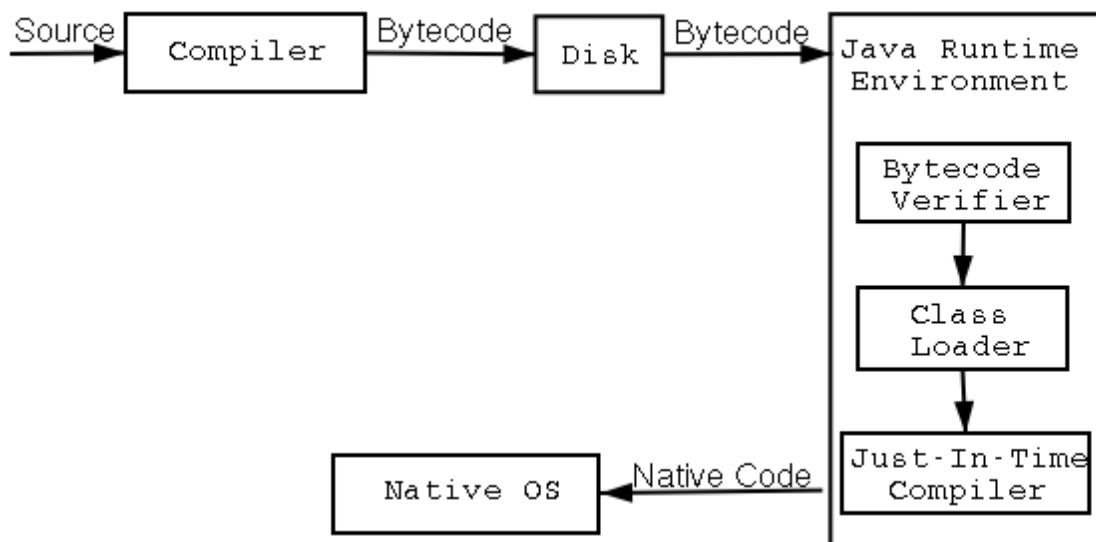
对整个程序进行分析
翻译成等价机器语言程序后执行

- 如Pascal, Fortran, C
- **优点：**执行效率高
- **缺点：**在运行中发现的错误
必须查找整个程序确定
移植性不好

■ Java程序从源文件创建到程序运行要经过两大步骤：

- 1、源文件由编译器编译成字节码（ByteCode）
- 2、字节码由Java虚拟机解释运行。

■ Java程序既要编译同时也要经过JVM的解释运行



JIT（just in time，即时编译）

编译技术的应用

- 很多机器学习的算法都会用Python来实现
- Python作为开源的编程语言，有多个版本的解释器，
 - C实现的CPython（官方提供）
 - Java实现的Jpython
 - Python实现的PyPy（Armin Rigo公司开发）
 - PyPy采用JIT（just-in-time）技术，对Python代码进行动态编译
- 在Kaggle的生物加速计竞赛（Kaggle 's Biometric Accelerometer Competition）中，用PyPy替换了CPython来对5650万行数据进行特征生成，数据处理时间由原来的**12个小时缩短为20分钟**

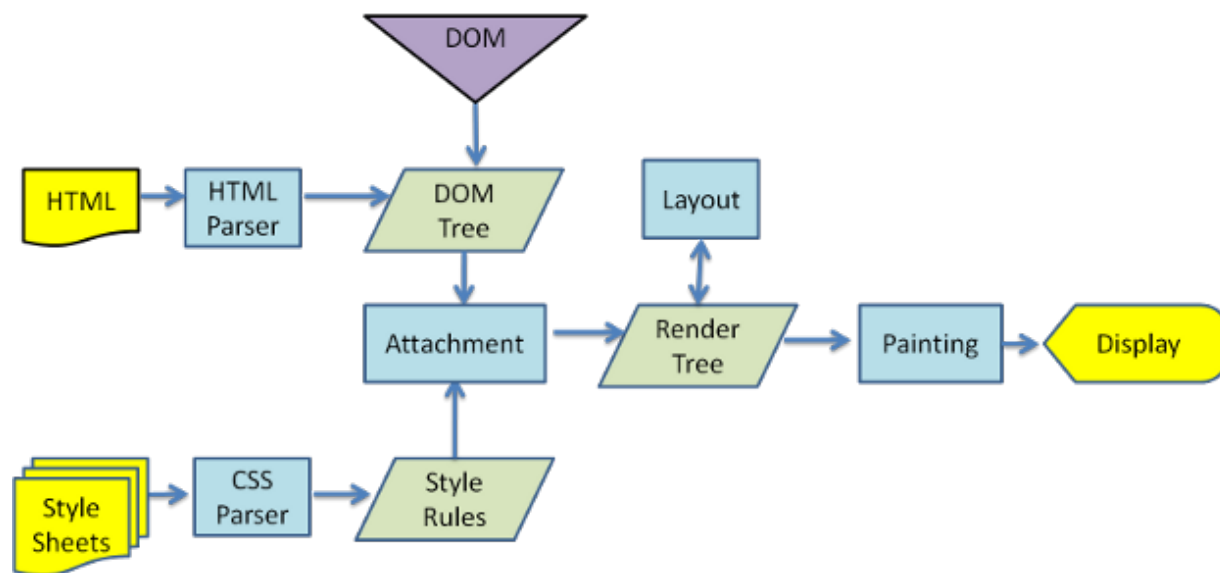
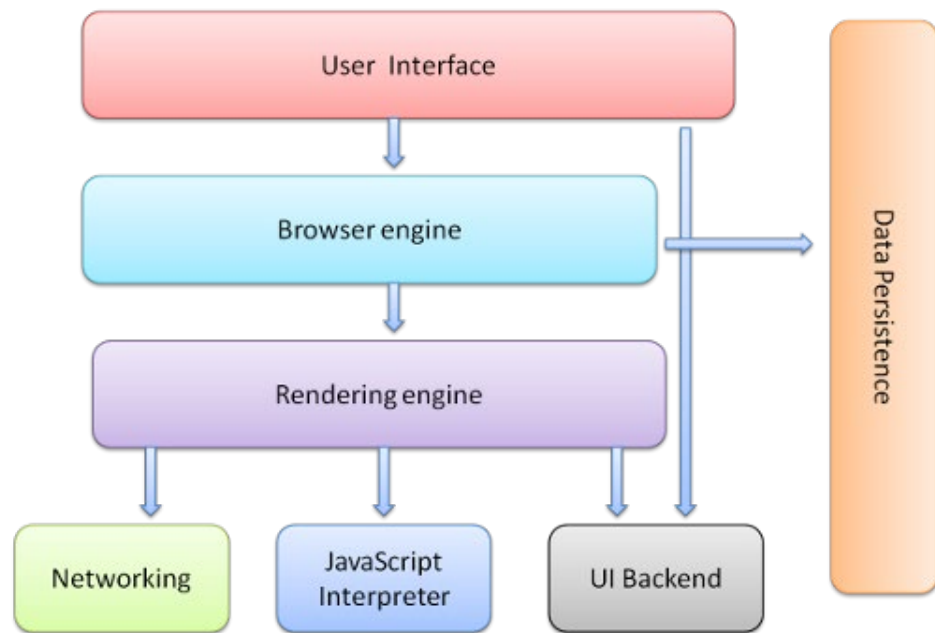
编译技术的应用

■ 文档转换工具

- ps2pdf
- Latex2html

■ Web 浏览器

- <https://www.oschina.net/p/webkit>

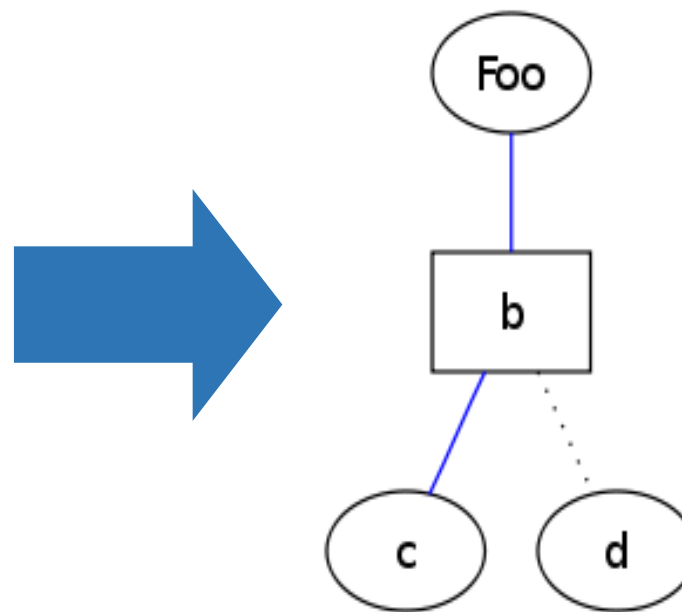


开源渲染引擎Webkit

编译技术的应用

■ 图形绘制工具Dot

```
graph graphname {  
    // label属性可以改变节点的显示名称  
    a [label="Foo"];  
    // 节点形状被改变了  
    b [shape=box];  
    // a-b边和b-c边有相同的属性  
    a -- b -- c [color=blue];  
    b -- d [style=dotted];  
}
```



- 语言转换工具
 - F2c (Fortran to C)
 - P2c (Pascal to C)

内容线索

√ 什么叫编译程序

2. 编译过程概述

3. 编译程序的结构

4. 编译程序的生成

5. 总结

一个类C源程序片段

```
void main(int b,int c)
{
    int a;
    a=b+(c-2);
}#
```

-
- ```
graph TD; A["void main(int b,int c) { int a; a=b+(c-2); }#"] --> B["1. 词法分析
2. 语法分析
3. 语义分析中间代码生成
4. 优化
5. 目标代码生成"]; B --> C["MOV EAX, [EBP+8+0]
SUB EAX, 2
MOV EBX, [EBP+8+4]
ADD EBX, EAX
MOV [EBP-4], EBX"]
```
1. 词法分析
  2. 语法分析
  3. 语义分析中间代码生成
  4. 优化
  5. 目标代码生成

?

```
MOV EAX, [EBP+8+0]
SUB EAX, 2
MOV EBX, [EBP+8+4]
ADD EBX, EAX
MOV [EBP-4], EBX
```

# 编译过程概述

- 掌握编译过程的五个基本阶段，是我们学习编译原理课程的基本内容，把编译的五个基本阶段与英译中的五个步骤相比较，有利于对编译过程的理解：

## 英译

1. 识别出句子中的一个单字
2. 分析句子的语法结构
3. 初步翻译句子的含意
4. 译文修饰
5. 写出最后译文

## 编译

1. 词法分析
2. 语法分析
3. 语义分析中间代码生成
4. 优化
5. 目标代码生成

- **词法分析程序又称扫描程序。**

- **任务：**读源程序的字符流、识别单词（也称单词符号，或简称符号），如标识符、整数、界限符等，并转换成内部形式。
- **输入：**源程序中的字符流
- **输出：**等长的内部形式，即属性字。

- **在词法分析阶段工作所依循的是语言的词法规则。**

- **描述词法规则的有效工具是正规式和有限自动机。**

- **方法：状态图；DFA；NFA**



# 词法分析示例

```
void main(int b,int c)
{
 int a;
 a=b+(c-2);
}#
```

词法分析后返回 (如右图):

单词值

单词类型

|         |           |
|---------|-----------|
| 1: void | VOID      |
| 1: main | MAIN      |
| 1: (    | LPAREN    |
| 1: int  | INT       |
| 1: b    | ID        |
| 1: ,    | COMMA     |
| 1: int  | INT       |
| 1: c    | ID        |
| 1: )    | RPAREN    |
| 2: {    | LBBRACKET |
| 3: int  | INT       |
| 3: a    | ID        |

单词值

单词类型

|      |           |
|------|-----------|
| 3: ; | SEMI      |
| 4: a | ID        |
| 4: = | ASSIGN    |
| 4: b | ID        |
| 4: + | PLUS      |
| 4: ( | LPAREN    |
| 4: c | ID        |
| 4: - | SUB       |
| 4: 2 | NUM       |
| 4: ) | RPAREN    |
| 4: ; | SEMI      |
| 5: } | RBBRACKET |
| 5: # | ENDFILE   |

## ■ 语法分析程序又称识别程序。

- **任务：**读入由词法分析程序识别出的符号，根据给定语法规则，识别出各个语法单位（如：短语、子句、语句、程序段、程序），并生成另一种内部表示。
- **输入：**由词法分析程序识别出并转换的符号。
- **输出：**另一种内部表示，如语法分析树或其它中间表示。

## ■ 语法规则通常用**上下文无关文法**描述。

## ■ 方法：递归子程序法、LR分析法、优先分析法。

## ■ 例: $a=b+(c-2)$

### 规则

$\langle \text{赋值语句} \rangle ::= \langle \text{标识符} \rangle \text{"="} \langle \text{表达式} \rangle$

$\langle \text{表达式} \rangle ::= \langle \text{表达式} \rangle \text{" + / - " } \langle \text{表达式} \rangle$

$\langle \text{表达式} \rangle ::= \text{"("} \langle \text{表达式} \rangle \text{"}")}$

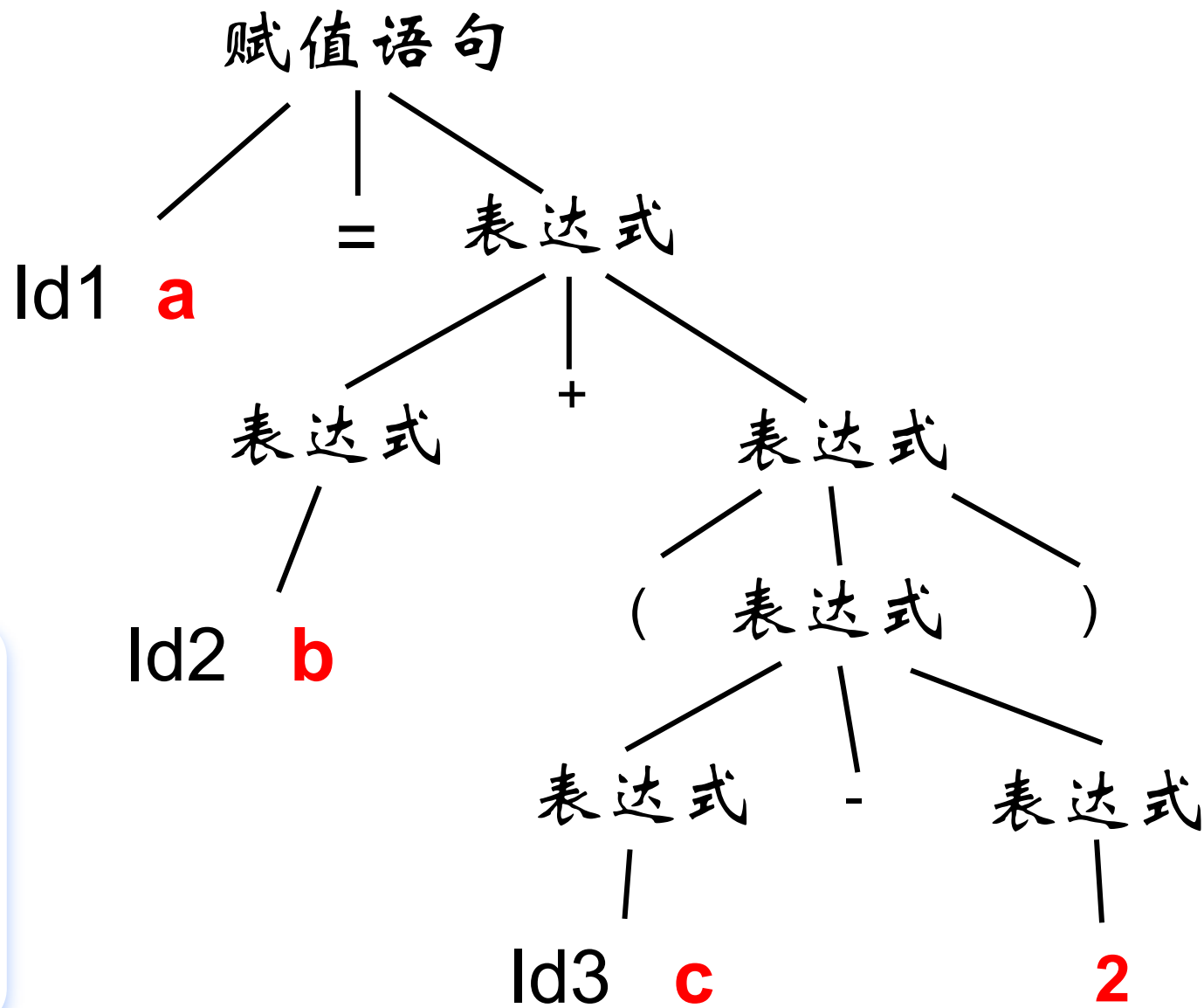
$\langle \text{表达式} \rangle ::= \langle \text{标识符} \rangle$

$\langle \text{表达式} \rangle ::= \langle \text{整数} \rangle$

# 语法分析

## ■ 例: $a=b+(c-2)$ 的语法树

其中: id1、id2、id3  
分别表示a、b、c的机  
器内部表示



- 对语法分析树或其他内部中间表示进行**静态语义检查**，如果正确则进行另一方面的工作，即进行中间代码的翻译。

- 按照语法树的层次关系和先后次序，逐个语句地进行语义处理。
- **主要任务：** 进行类型审查，审查每个算符是否符合语言规范，不符合时应报告错误。
  - 类型匹配
  - 类型转换

```
void main(int b,int c)
{
 int a;
 a=b+(c2-2);
}#
```

# 中间代码

- **中间代码是一种独立于具体硬件的记号系统，或者与现代计算机的指令形式有某种程度的接近，或者能比较容易地变换成机器指令。**
  - **任务：**将各类语法单位，如“表达式”、“语句”、“程序”等翻译为中间代码序列。
  - **输入：**句子。
  - **输出：**中间代码序列。
- **中间代码的形式：**常见的有四元式、三元式和逆波兰式等
- **方法：**语义子程序；DAG图，语法制导翻译

## ■ 四元式的形式为：

(算符, 运算对象1, 运算对象2, 结果) ;

id1

id2

id3

## ■ 对于源程序 $a = b + (c - 2)$ 可以生成如下所示的四元式：

| ID   | 中间代码          |
|------|---------------|
| main |               |
| 0    | - , c, 2, T0  |
| 1    | + , b, T0, T1 |
| 2    | = , T1, , a   |

id1、id2、id3分别表示a、b、c的机器内部表示  
T0,T1是临时生成的名字, 表示中间运算结果。

T0,T1, 是临时变量



## 例1

```
void main(int b,int c)
{

 int a;

 a=b+(c-2);

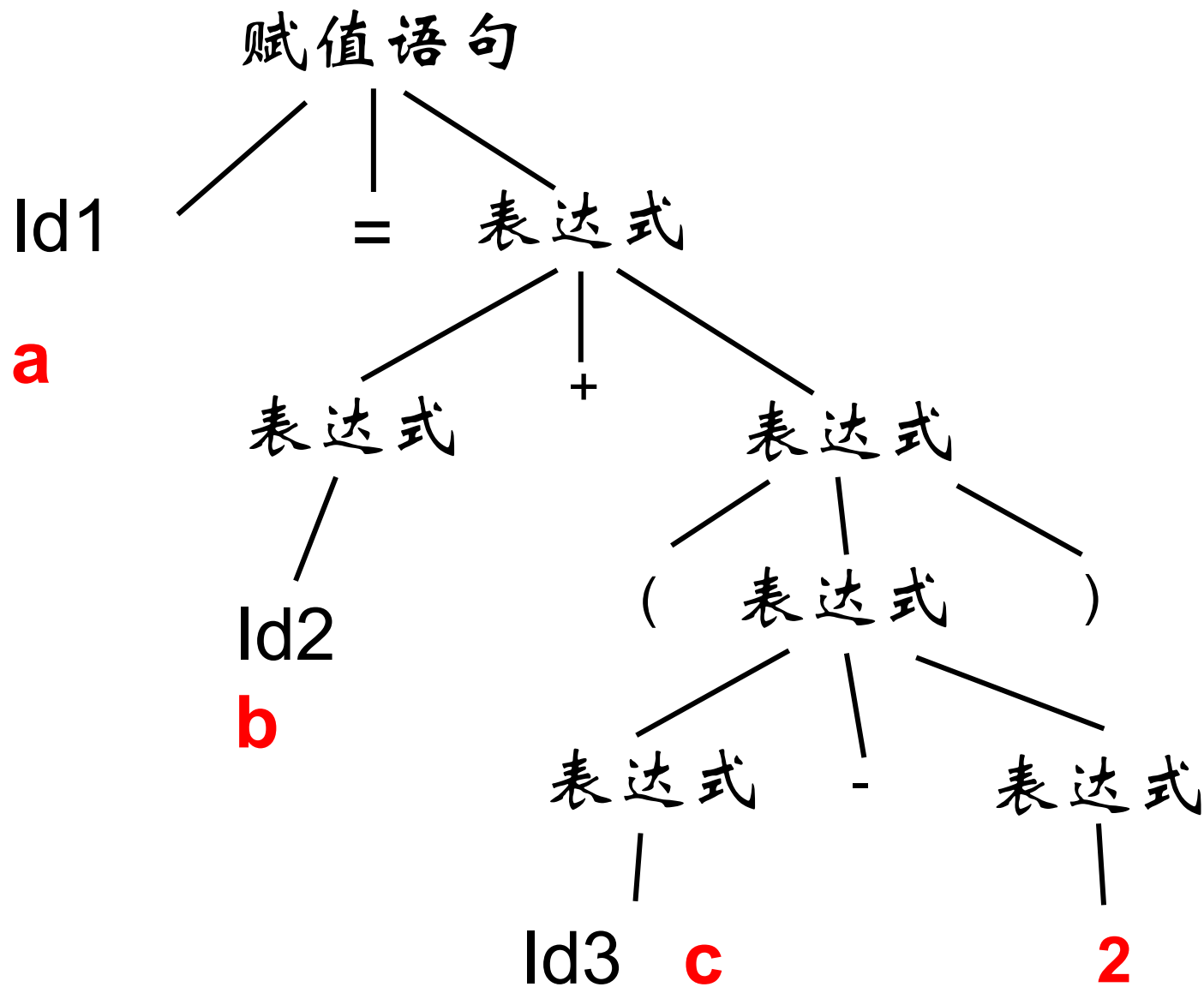
}#
```

# 语义分析

## $a = b + (c - 2)$ 的语法树

其中: id1、id2、id3分别表示a、b、c的机器内部表示

| ID   | 中间代码          |
|------|---------------|
| main |               |
| 0    | - , c, 2, T0  |
| 1    | + , b, T0, T1 |
| 2    | = , T1, , a   |



## 例2

### ■ C语言的源程序

$a = b * c + b * d$

### 三地址序列 (赋值语句形式的四元式)

(1)  $t1 := b * c$

(2)  $t2 := b * d$

(3)  $t3 := t1 + t2$

(4)  $a := t3$

## 例3

### ■ C语言的源程序

if ( $a \leq b$ )

$a = a - c;$

$c = b * c;$

翻译成

四元式

100 ( $j \leq, a, b, 102$ )

101 ( $j, \_, \_, 104$ )

102 ( $-, a, c, t1$ )

103 ( $=, t1, \_, a$ )

104 ( $*, b, c, t2$ )

105 ( $=, t2, \_, c$ )

- **优化的任务在于对前段产生的中间代码进行加工，把它变换成功能相同，但功效更高的优化了的中间表示代码，以期在最后阶段产生更为高效（省时间和空间）的代码**
- **优化所依循的原则是程序的等价变换规则**
- **其方法有：公共子表达式的提取、循环优化、删除无用代码等。**

# 优化举例

- GCC的不同优化等级，代表着不同的优化程度。  
-O0、-O1、-O2、-O3代码优化程度依次递增

```
test.c
int main() {
 int i,j;
 i=5+6-7-4;
 while(i<1000*1000*1000)
 {
 i++;
 j=1+2+3+4;
 }
 return 0;
}
```

- 用gcc -O0 test.c -o test0生成可执行文件test0，运行时间**1.367s**
- 用gcc -O1 test.c -o test1生成可执行文件test1，运行时间**0.241s**
- 用gcc -O2 test.c -o test2生成可执行文件test2，运行时间**0.000s**  
(几乎忽略不计)

# 目标代码生成

- 这一阶段的任务：把中间代码（或经优化处理后）变换成特定机器上的低级语言代码。它有赖于硬件系统结构和机器指令含义。

## ➤ 与机器相关

| ID   | 中间代码          |
|------|---------------|
| main |               |
| 0    | - , c, 2, T0  |
| 1    | + , b, T0, T1 |
| 2    | = , T1, _, a  |



```
MOV EAX, [EBP+8+0]
SUB EAX, 2
MOV EBX, [EBP+8+4]
ADD EBX, EAX
MOV [EBP-4], EBX
```

# 内容线索

√ 什么叫编译程序

√. 编译过程概述

**3. 编译程序的结构**

**4. 编译程序的生成**

**5. 总结**

# 编译程序的结构





## ■ 编译程序涉及的表格有

|       |            |       |             |
|-------|------------|-------|-------------|
| 符号名表  | 常量名、<br>质、 | 循环表   | 名、过程名、<br>性 |
| 常数表   | 各种类        | 等价名表  |             |
| 标号表   | 标号的        | 公用链表  |             |
| 入口名表  | 过程的        | 格式表   | 位置          |
| 过程引用表 | 外部调        | 中间代码表 | 位置          |

## ■ 在编译程序使用的表格中最重要的是符号表

- 记录源程序中使用的名字(标识符)
- 收集每个名字的各种属性信息
  - 类型、作用域、分配存储信息

| 名 字 | 种 类 | 类 型  | 层 次 | 偏移量 |
|-----|-----|------|-----|-----|
| m   | 过 程 |      | 0   |     |
| a   | 变 量 | real | 1   | d   |
| b   | 变 量 | real | 1   | d+4 |
| c   | 变 量 | real | 1   | d+8 |

# 出错处理

- 程序中的错误可分为**语法错误**和**语义错误**两类。
- 语法错误可在词法分析和语法分析阶段查出来。
- 例如，下列错误都属于语法错误：

(1) 缺少分隔符号：DIMENSION A(10) B(10)

(2) 保留字拼写错误：DEMENSION A(10)

(3) 括号不配对：

WRITE (6,10) (A (1, J), J = 1, 10), I = 1, 10)

此外还有多余分隔符号，无循环终结语句等等。

# 语义错误

- 语义错误有些可在**编译时**查出来，有些则需在**运行时**才能查出来。
- 典型的语义错误：
  - 标识符没有说明就使用；
  - 标号有引用而无定义；
  - 形式参数和实在参数结合时在类型、个数、位置等方面不一致等等；
- 这些错误可在编译时查出来，而另一些错误如下标越界、运算溢出、调用某些标准函数时自变量的值不符合要求等，则需要到程序运行时才能查出来。

## ■ 一个好的编译器应该：

全

最大限度发现错误

准

准确指出错误的性质和发生地点

局部化

将错误的影响限制在尽可能小的范围内

若能**自动校正错误**则更好，但其**代价非常高**

# 编译阶段的组合

- 编译程序可以从逻辑上分成几个阶段，对于各个阶段的划分仅仅是指其逻辑结构，而在具体实现时，经常是将几个阶段组合在一起。例如，可以将各部分组合成前端和后端。

主要与源语言有关

## 编译过程

前端：词法分析、语法分析、语义分析、中间代码生成、优化工作。

后端：目标代码生成、出错处理、符号表操作。

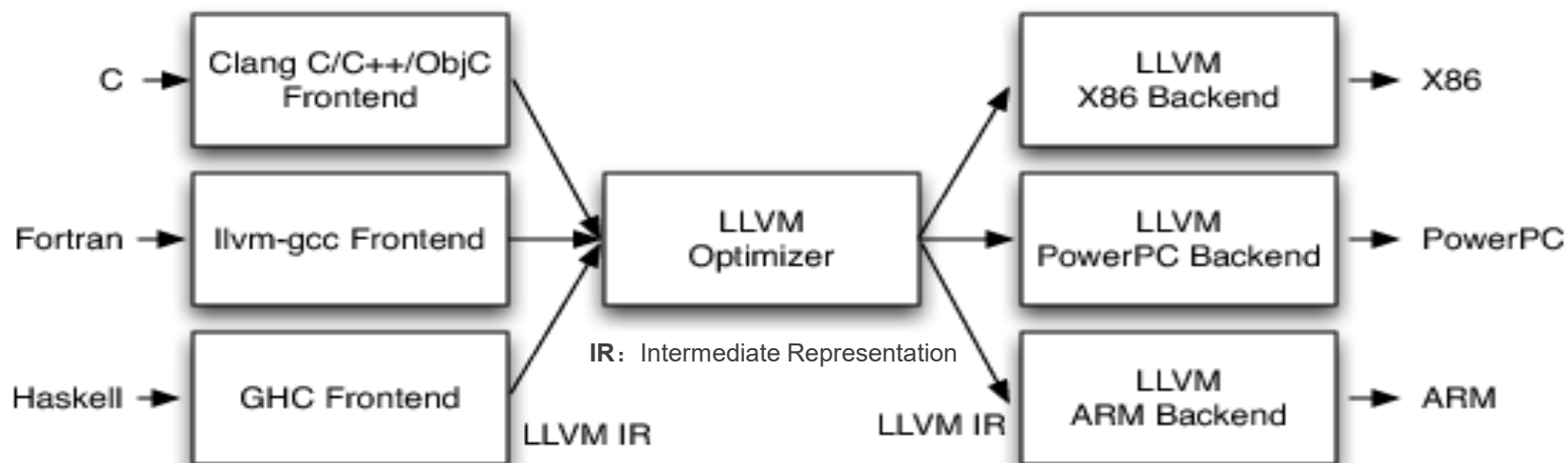
主要与目标代码有关

## ■ GCC (GNU Compiler Collection, GNU编译器集合)

➤ 1987 年, Richard Stallman

## ■ LLVM (low level virtual machine)

➤ 2003年, Chris Lattner



# 并发程序的例子

```
int x; //共享变量x
void* tprocess1(void* args) {
 x := 42;
}
void* tprocess2(void* args) {
 x := x + 1;
}
int main() {
 x = 0;
 pthread_t t1;
 pthread_t t2;
 pthread_create(&t1, NULL, tprocess1, NULL);
 pthread_create(&t2, NULL, tprocess2, NULL);
 pthread_join(t1, NULL);
 pthread_join(t2, NULL);
 assert(x >= 42);
}
```

## 测试方法

**执行10000次该pthread程序出现**

- **9992次assert(x >= 42)正常执行**
- **8次assert(x >= 42)报错**



# 并发程序的例子

```
int x; //共享变量x
void* tprocess1(void* args) {
 x := 42;
}
void* tprocess2(void* args) {
 x := x + 1;
}
int main() {
 x = 0;
 pthread_t t1;
 pthread_t t2;
 pthread_create(&t1, NULL, tprocess1, NULL);
 pthread_create(&t2, NULL, tprocess2, NULL);
 pthread_join(t1, NULL);
 pthread_join(t2, NULL);
 assert(x >= 42);
}
```

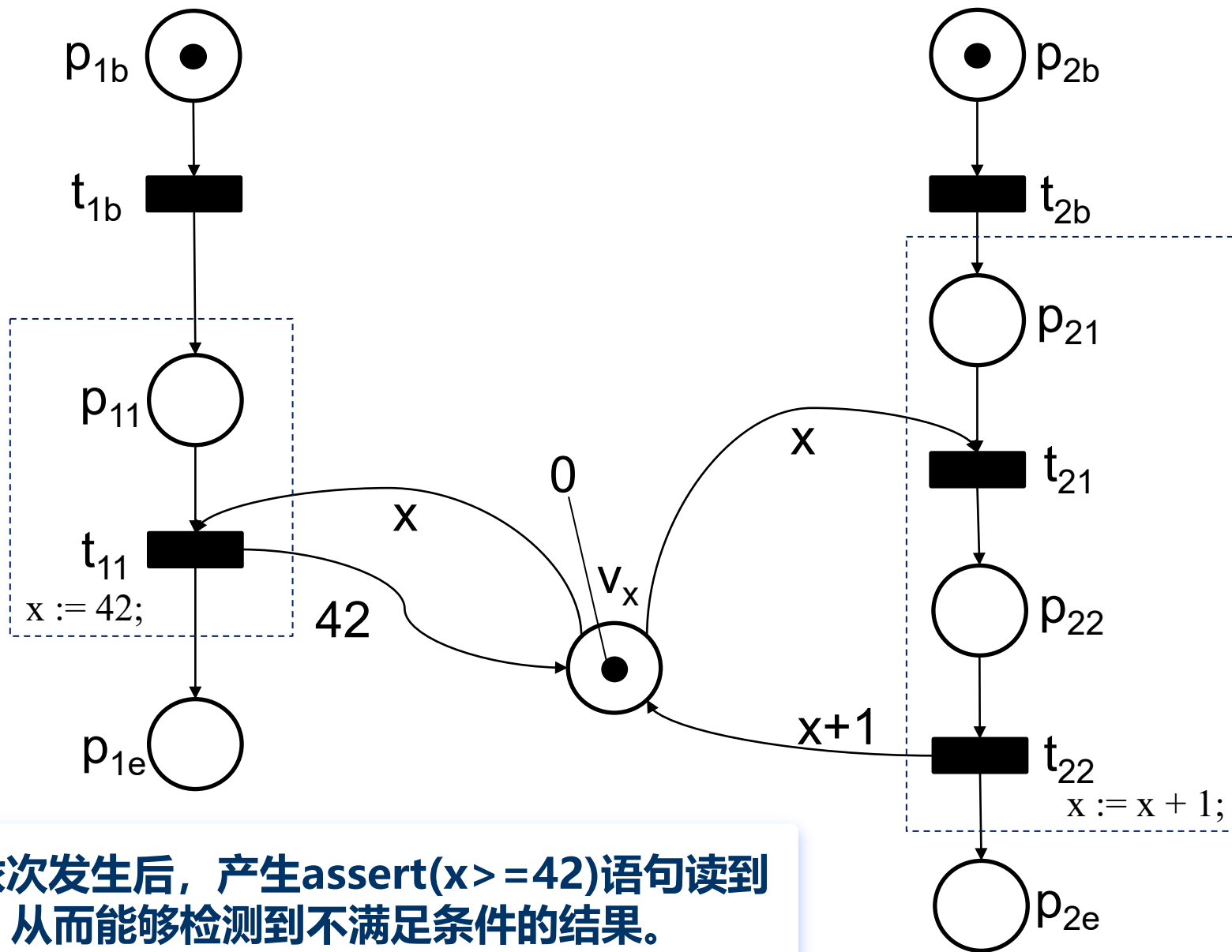
- 若语句都是具有原子性的，可能的结果如下：
  - 若  $x := 42$ ; 在  $x := x + 1$ ; 之前执行，`assert(x >= 42)` 语句读到的  $x$  为 43。
  - 若  $x := x + 1$ ; 在  $x := 42$ ; 之前执行，`assert(x >= 42)` 语句读到的  $x$  为 42。
- 这两种情况下，`assert` 语句中的  $(x >= 42)$  条件都是满足的。

- 但是实际上  $x := x + 1$ ; 在实际执行中并非原子的，其中从内存中读  $x$  的当前值的操作与将  $x + 1$  的计算结果写到内存的操作可能与另一线程的写操作交织，使得 `assert(x >= 42)` 语句读到的  $x$  为 1，不满足条件。

# 形式化验证

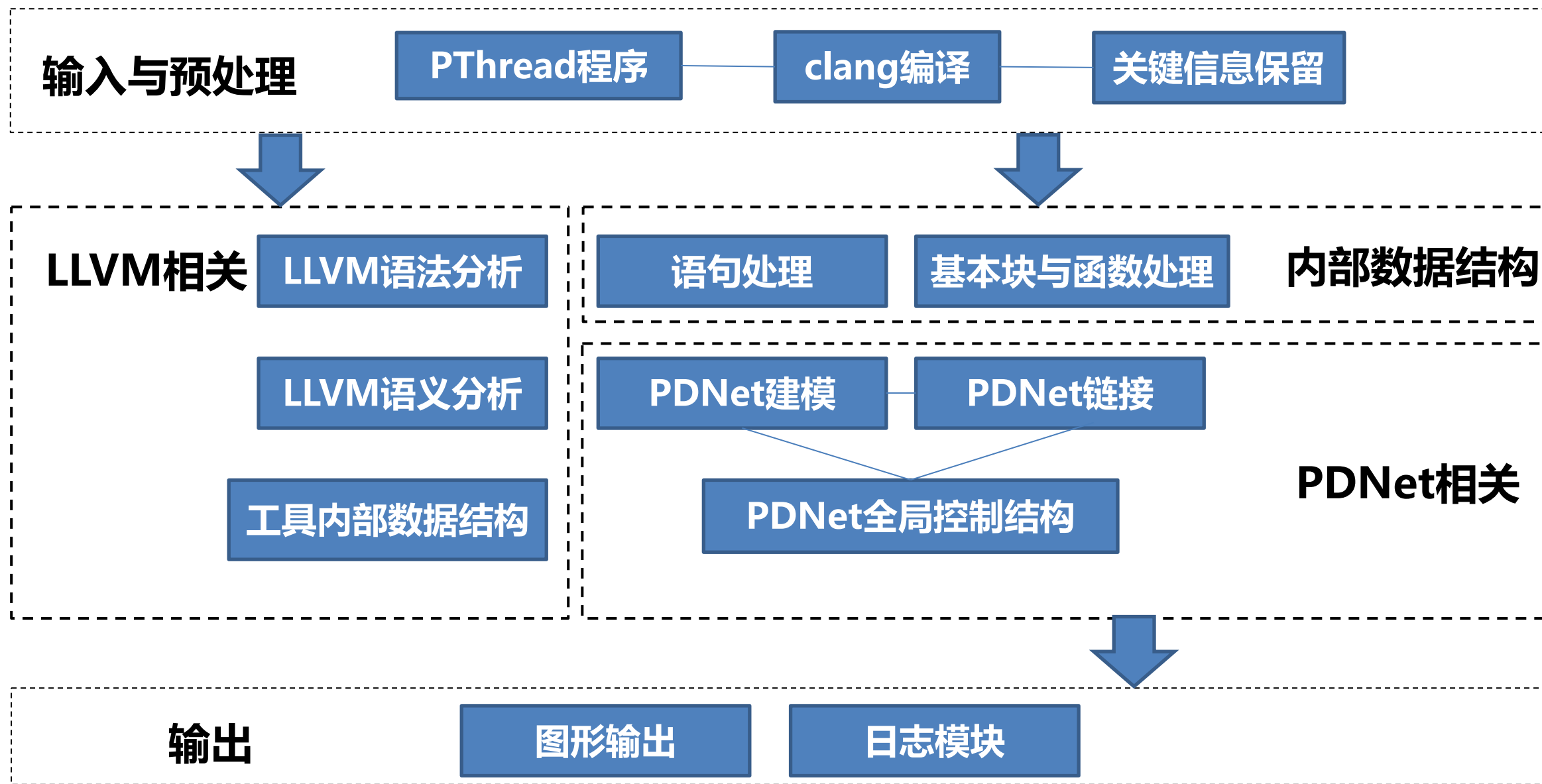
- 由于大量的不确定交织的存在，并发程序的性质验证存在更大的挑战。
- 形式化验证能够模拟并发程序所有可能的执行，考虑任意线程交错，这一现象在程序测试和定理证明中都很难捕捉到。
- 对程序员编写的代码，即由高级编程语言构成的代码，进行自动的形式验证，保证其正确性、安全性等是极其重要的。
- 自动化工具能够直接对高级编程语言进行模型检测，而无需模型检查中通常需要的专业工作是期望的。

# 基于Petri网的程序模型

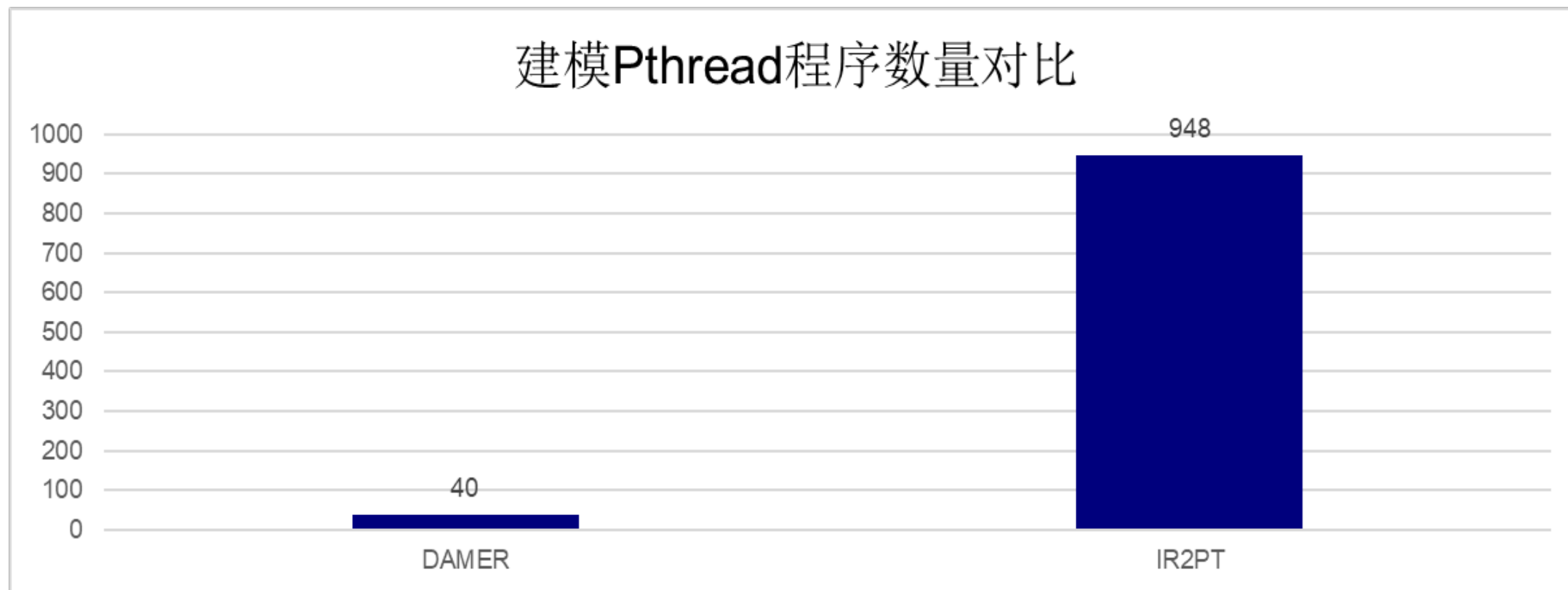


变迁 $t_{21}$   $t_{11}$   $t_{22}$ 依次发生后，产生`assert(x >= 42)`语句读到的 $x$ 为1的情况，从而能够检测到不满足条件的结果。

# 基于IR的PDNet自动化建模



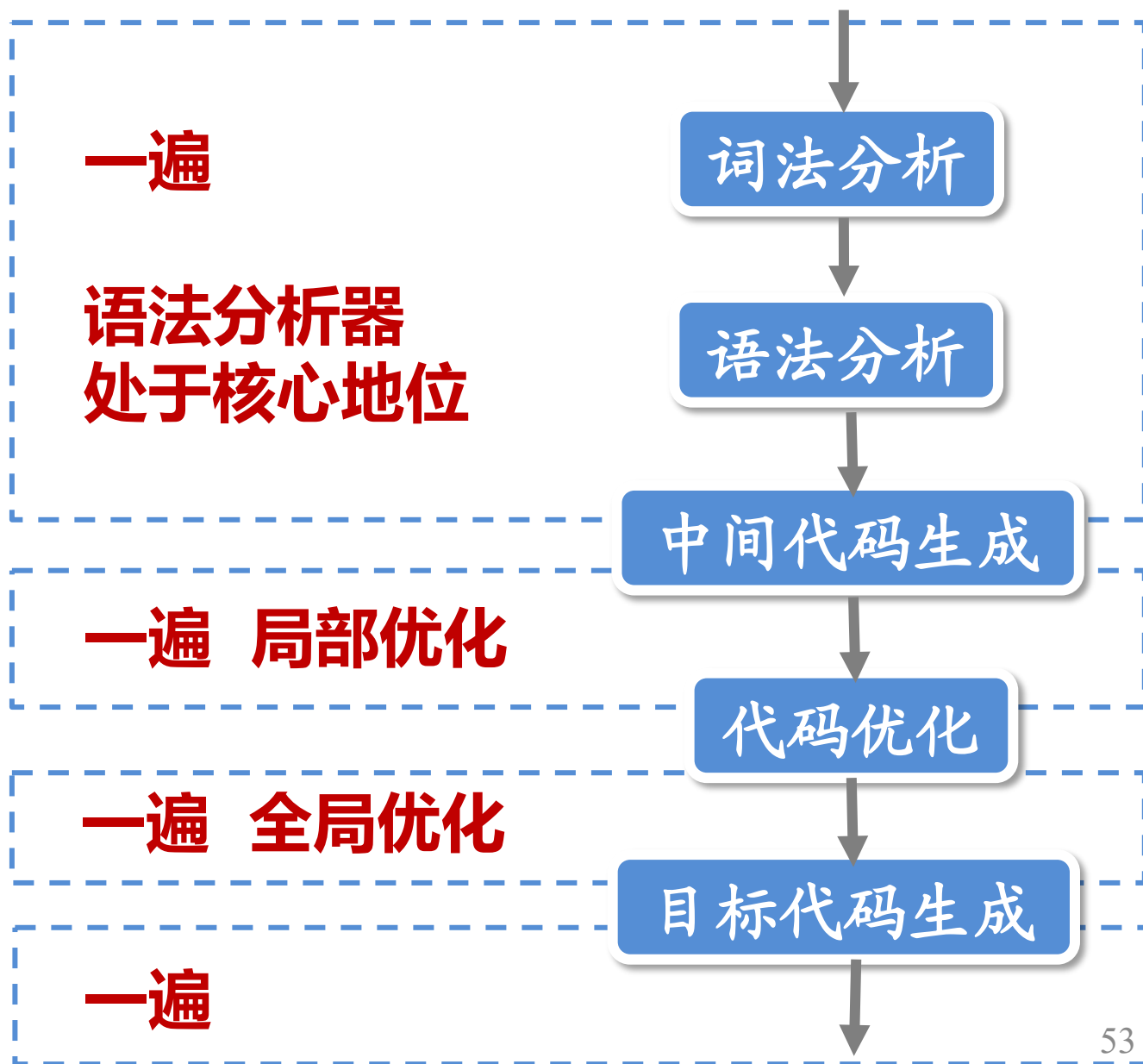
# 基于IR的PDNet自动化建模



**实验测试集共有1044个Pthread程序测试用例，可以看到我们的方法实现测试点覆盖率达90.8%。**

# 遍 (Pass)

- 对源程序或源程序的中间结果从头到尾扫描一次，并做相关处理，生成新的中间结果或目标程序的过程。
- “遍”是处理数据的一个完整周期，每遍工作从外存上获得前一遍的中间结果（源程序），完成它所含的有关工作之后，再把结果记录于外存。



# 遍的次数和效果

- 一个编译程序可由一遍、两遍或多遍完成。每一遍可完成不同的阶段或多个阶段的工作

从时间  
和空间  
角度看

多遍编译

—— 少占内存，多耗时间

一遍编译

—— 多占内存，少耗时间

# 内容线索

√ 什么叫编译程序

√. 编译过程概述

√. 编译程序的结构

**4. 编译程序的生成**

**5. 总结**



# 编译程序实现语言

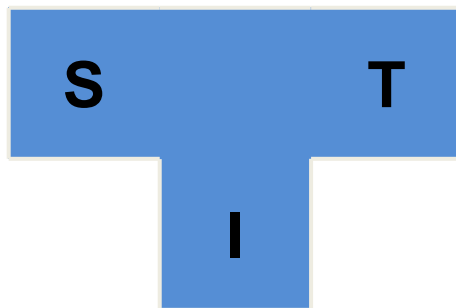
- 机器语言

- 汇编语言

- 充分发挥各种不同硬件系统的效率
- 满足各种不同的具体要求

- 高级语言

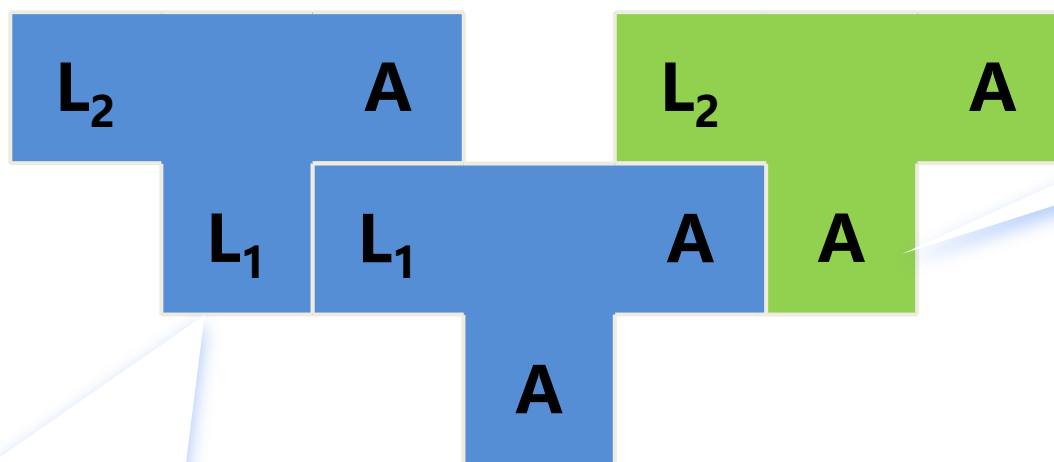
- 大大节省程序设计的时间
- 构造出来的编译程序易于阅读、维护和移植



## ■ 其中:

- S: 源语言(程序), Source language(program)
- T: 目标语言(程序), target/object language(program)
- I: 实现语言, implementation language

## 示例

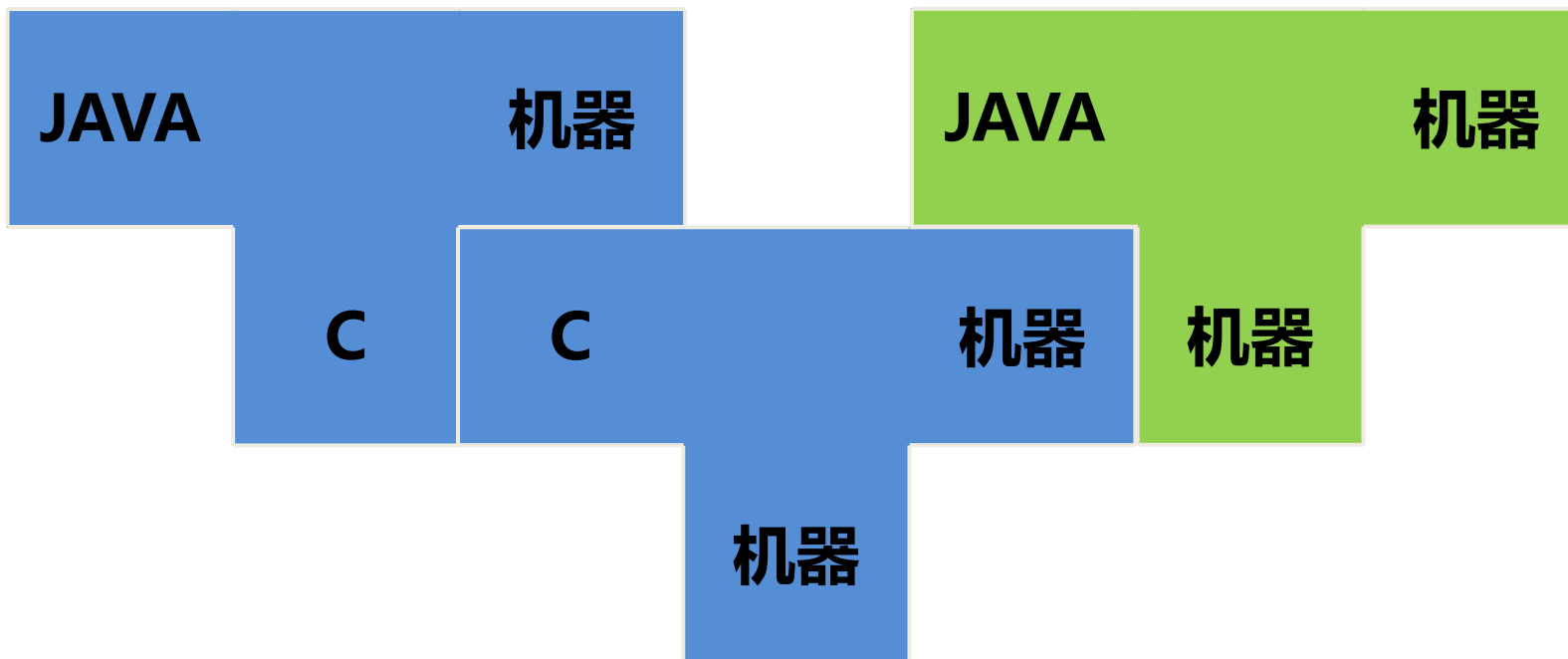


**A机器代码实现  
的 $L_2$ 编译程序**

**用 $L_1$ 语言编写另一种高级语言 $L_2$ 的编译程序**

**A机器上已有一个用A  
机器码实现的某高级语言 $L_1$ 的编译程序**

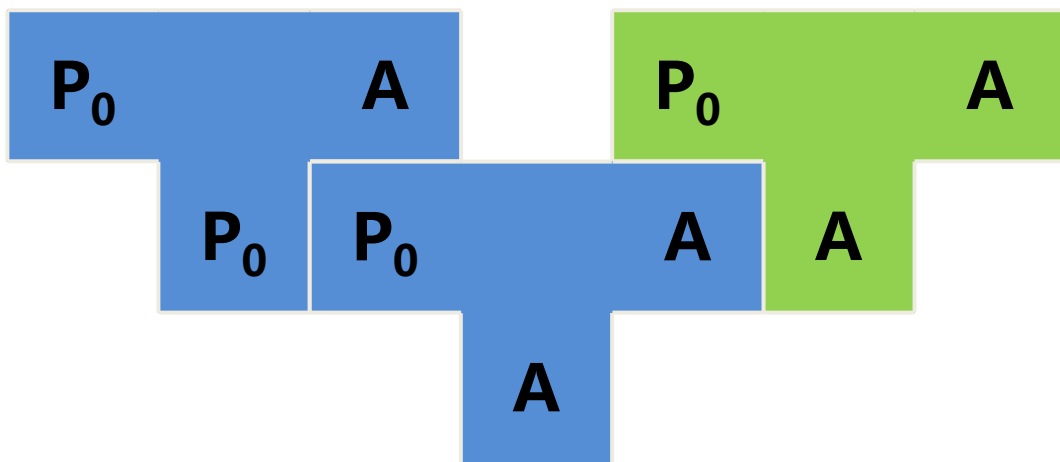
## 示例



# 编译程序的生成技术

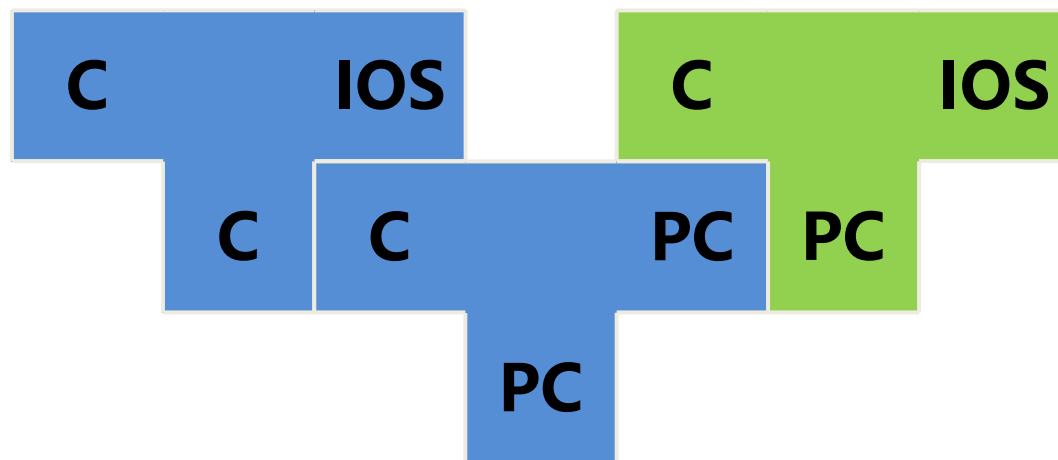
- 自编译
- 交叉编译
- 自展
- 移植

- 用某种高级语言书写自己的编译程序称为自编译。
  - 例如,假定A机器上已有一个PASCAL语言编译程序,则可用PASCAL语言编写一个功能更强的PASCAL语言编译程序,然后借助于原有的编译程序对新编写的PASCAL编译程序进行编译,从而得到一个能在A机器上运行的功能更强的PASCAL编译程序。



# 交叉编译

- 用x机器上的编译程序产生可在y机器上运行的目标代码称为交叉编译。
- 例如, 若PC机器上已有C语言编译程序,则可用PC机器中的C语言书写一个编译程序, 该编译程序的源程序是C语言程序,而产生的目标程序则是基于IOS终端的, 即产生在IOS终端上执行的低级语言程序。



- 适用于嵌入式系统、移动端开发
- 上述两种方法假定已有一个编译程序, 若没有, 则可采用自展或移植法。

- 首先确定一个非常简单的核心语言 $L_0$ ，然后用机器语言或汇编语言书写出其编译程序 $T_0$ ；
- 到 $L_1$ ， $L_0 \subset L_1$ ，并用 $L_0$ 编写 $L_1$ 的编译程序 $T_1$ (自编译)；
- 然后再把语言 $L_1$ 扩充为 $L_2$ ， $L_1 \subset L_2$ ，并用 $L_1$ 编写 $L_2$ 的编译程序 $T_2$ ；
- .....

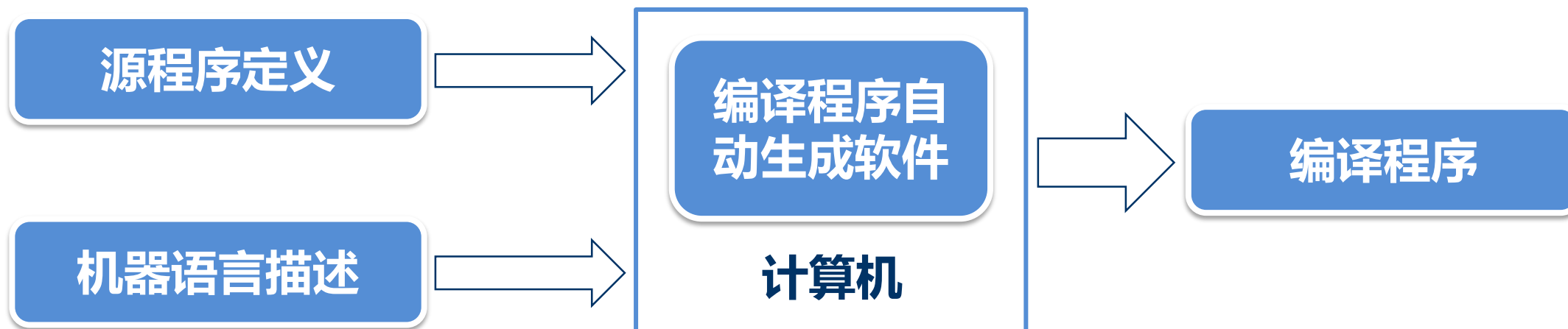
这样不断扩展，直到完成所要求的编译程序为止。



- **移植是指A机器上的某种高级语言的编译程序稍加改动后能在B机器上运行。**
- **一个程序若能较易地从A机移到B机上运行, 则称该程序可移植。**

# 自动编译

- 自动生成编译程序的软件工具, 只要把源程序的定义及机器语言的描述输入到该软件中, 就能自动生成该语言的编译程序。



- 目前的编译程序自动生成系统, 如LEX和YACC等。

# 编译程序的分类

- 依据编译程序的不同用途和侧重，可分类为：
  - 诊断型编译程序
  - 优化型编译程序
  - 交叉型编译程序
  - 可变目标型编译程序

# 编译技术的发展

- 在1954年至1957年期间，IBM的John Backus带领的一个研究小组对FORTRAN语言及其编译器的开发
  - 将算术公式翻译成机器代码。
- Noam Chomsky 开始了自然语言结构的研究。他的发现最终使得编译器结构异常简单
- 70年代，有穷自动机和形式语言的研究，促进了编译器的发展
  - Steve Johnson为Unix系统编写了**Yacc** (yet another compiler-compiler) 。
  - Mike Lesk为Unix系统开发的 **Lex**

# 编译技术的发展

## 1990年代：开源工具链与跨平台编译的兴起

### ■ GCC (GNU Compiler Collection)

- 首个开源、多语言 (C/C++等)、多平台支持的编译器。
- 插件化架构，奠定可扩展编译框架基础。
- 推动Linux生态崛起 (如Linux内核依赖GCC编译)。

### ■ LLVM (Low Level Virtual Machine)

- 模块化设计 (前端Clang、中端优化器、后端代码生成)。
- 解耦编译器开发，加速优化算法迭代 (如苹果Swift语言依赖LLVM)。
- 支持JIT编译，为后续动态语言优化铺路。

### ■ Java虚拟机 (JVM)

- “Write Once, Run Anywhere” 字节码 (.class文件) 机制。
- JIT技术初现：HotSpot虚拟机动态优化热点代码 (解释执行→即时编译)。

## 21世纪——性能优化与AI赋能

### ■ JIT（即时编译）与动态语言优化

- **V8引擎（2008）：**
  - 关键技术：隐藏类（Hidden Class）、内联缓存（Inline Cache）
  - 影响：Chrome浏览器、Node.js生态的基石
- 其他案例：PyPy（Python JIT）、LuaJIT

### ■ AI辅助编译

- 机器学习预测代码优化策略（如循环展开、向量化）
- 自动调优编译器参数（如AutoTVM优化深度学习算子）

## ■ 20世纪60年代

- 董韫美院士、杨芙清院士分别在中科院和北大领导研究组开发编译器，面向的高级语言是 ALGOL和FORTRAN，目标机是国产机。
- 在改革开放前，由于国家需要，中科院、国防科大、江南计算所、北大等单位一直在研制国产计算机，包括大型机和高性能计算机（如向量机、并行机），相应的也在研制高级语言编译器。
- 中科院计算所以董韫美院士领导的研究组先后开发了119机、109机的类 ALGOL语言编译器 BCY。国防科大开发了向量编译器和向量识别器。

## ■ 20世纪90年代

- 90年代ACTGroup承担科学院重大项目，国家攻关项目，863项目，以及国际合作项目，先后开发了共享内存多处理机的并行识别器，分布式内存多处理机的并行识别器，SIMD芯片和VLIW芯片的并行优化C编译器。将编译技术与图形学结合，ACTGroup还推出了集成化、可视化的并行编程环境。
- ACTGroup在先进编译技术和并行编程环境方面的研究工作获国内外专家高度评价，国际著名学者评价此研究组居编译领域的世界先进行列。



## ■ 20世纪90年代

- 90年代，国内主要以研制并行机为主，相应的并行编译器研制也在国内开展起来。
- 清华大学在软流水优化技术上做了很优秀的研究工作，开发了交互式并行化系统 TIPSExplorer。
- 北京大学在HPF (High Performance Fortran) 编译器方面做了多年工作，取得很好的研究成果。
- 此外，国防科大、江南计算所等单位也都有从事并行编译技术研究。
- 随着芯片研制，国内还有若干单位也在开展基于GCC生成面向特定芯片的编译器工作。

## ■ 热点产品——华为方舟编译器

- 是华为公司专门为软件厂商研发的**统一编程平台**，包含编译器、工具链、运行时等关键部件。该编译器支持多种编程语言、多种芯片平台的联合编译与运行，能够有效解决安卓程序“边解释边执行”的低效率问题。
- 是华为推出的**首个完全自主研发的编译器平台**，于2019年8月31日正式上线并部分开源。
- 目前仍在持续更新。



## ■ 热点产品——华为毕昇编译器

- 是华为公司针对鲲鹏硬件平台的高性能编译器，是一款高性能、高可信及易扩展的编译器工具链，增强和引入了多种编译优化技术，支持C/C++/Fortran等编程语言。提供深度优化的编译技术，增强多核并行化，自动矢量化等，大幅提升指令和数据吞吐量，针对某些应用场景进行优化，尤其在高性能计算（HPC）场景下能获得更优的性能收益。
- 基于开源LLVM 10.0.1版本开发，并进行了优化和改进，于2020年9月30日正式上线。
- 当前最新版本为 2.1.0，于 2021 年 12 月 30 日发布。



## ■ 热点产品——腾讯Yadcc编译器

- 是腾讯公司研发并开源的C++分布式编译系统，针对实际的工业生产环境进行了性能、可靠性、易用性等方面优化，可以利用几百乃至1000+核同时编译，大大加快构建速度。通过创建符号链接至Yadcc客户端以截获编译器调用，再将任务分发至分布式编译机编译，降低本地负载。用于支撑腾讯广告的日常开发及流水线，目前在腾讯1700+核的集群中每天编译300,0000+个目标文件，产出约3~5TB。
- 调研学习业界开源编译加速系统后，根据实际工业生产场景自行设计，于2021年6月正式对外开源。

## ■ 热点产品——阿里BladeDISC编译器

- 是阿里巴巴集团自主研发并开源的深度学习编译器，旨在为用户提供通用、透明、易用的深度学习性能优化能力，支持主流的机器学习框架和主流硬件。在架构层面根本地解决了深度学习领域具有动态尺寸 Tensor 的优化难题，提供了多种灵活的部署方案，包括插件模式集成、独立运行与AOT 编译。是业内较早投入实际业务应用的编译器。
- 基于MLIR编译器框架开发，于2022年1月28日正式开源。

# 热点产品对比

| 比较   | 华为方舟                     | 华为毕昇                   | 腾讯Yadcc                       | 阿里BladeDISC                          |
|------|--------------------------|------------------------|-------------------------------|--------------------------------------|
| 简介   | 为软件厂商研发的统一编程平台           | 针对鲲鹏硬件平台的高性能编译器        | 支撑腾讯广告的日常开发及流水线的编译器           | 投入公司实际业务应用的自用编译器                     |
| 特色   | 能够有效解决安卓程序“边解释边执行”的低效率问题 | 针对高性能计算场景优化，能获得更优的性能收益 | 针对分布式工业生产环境进行了性能、可靠性、易用性等方面优化 | 在架构层面根本地解决了深度学习领域具有动态尺寸 Tensor 的优化难题 |
| 基础框架 | 完全自研                     | 基于LLVM                 | 自行设计                          | 基于MLIR                               |
| 开源时间 | 2019.8.31                | 2020.9.30              | 2021.6                        | 2022.1.28                            |

# 编译技术发展趋势：智能、异构、安全、量子

## 1. AI驱动的编译优化

- **自动调优：**
  - 用LLM预测最优的Pass（phase ordering problem, 优化阶段排序）序列（如-O1, -O2组合）
- **窥孔优化：**
  - LLM自动发现并生成缺失的指令替换规则（如将复杂的乘加指令简化为单条指令）
- **性能预测：**
  - 用GNN（图神经网络）分析IR的图结构，预测不同硬件（GPU/TPU）上的执行延迟，替代传统基于循环嵌套深度的静态模型
- **Bug查找与修复：**
  - LLM理解编译器源码（如GCC百万行代码），自动定位并发掘深层的逻辑缺陷（Miscompilation）

## 2. 异构计算编程

- **MLIR (Multi-Level IR)：**
  - 取代单层LLVM IR，提供多级抽象（Tensor, Affine, LLVM IR），允许在高级别做领域特定优化（如AI张量分解），再逐步降级到硬件。
- **统一编程模型：**
  - C++标准引入offload\_to(gpu\_device)属性，编译器自动将Lambda函数编译为SPIR-V (Vulkan) 或PTX (NVIDIA) 或HSACO (AMD)。
- **Axe Layout Abstraction：**
  - 提出统一的“数据布局”抽象，用一套语法描述数据在GPU线程块、AI加速器（TPU）片内存存储的分布，解决手工优化移植难的痛点。

## 3. 安全与形式化验证

- **内存安全：**
  - 在IR级别插入静态检查（如Stack Canaries, Bounds Checking），或通过类型系统（如Rust编译器的Borrow Checker）在编译时消除Use-after-free。
- **验证优化正确性：**
  - 使用定理证明器（Coq, Lean）验证每个优化Pass（如循环展开、内联）是**语义保持 (Semantics-Preserving)** 的，即优化不会改变程序行为。
- **Null指针静态分析：**
  - 基于流敏感分析（Flow-sensitive），默认将指针视为nonnull（根据Google数十亿行代码统计，非空指针占比高达6:1），在编译期报错而非运行时崩溃。

## 4. 量子编译

- **量子比特映射 (Qubit Mapping)：**
  - 将逻辑量子比特分配到物理芯片上相邻的位置，减少SWAP操作（昂贵）。使用图匹配和启发式搜索。
- **脉冲级优化 (Pulse-level)：**
  - 不再将门视为原子操作，而是编译为具体的微波脉冲波形，利用物理效应（如CNOT门）减少门数量。
- **变分量子算法 (VQA)：**
  - 编译器与经典优化器（如梯度下降）协同，自动计算参数梯度并生成混合经典-量子代码。

# 编译方向推荐论文

## 一、经典基石（必读，理解现代编译器为何如此设计）

- **Optimizing Compilers for Modern Architectures** - Allen & Kennedy : 系统化阐述了依赖分析（Dependence Analysis）和循环变换（Loop Transformations）的数学基础。
- **Efficiently Computing Static Single Assignment Form and the Control Dependence Graph** (1991) - Cytron et al. : 提出了高效计算SSA（静态单赋值）形式的经典算法。
- **The Program Dependence Graph and Its Use in Optimization** (1987) - Ferrante et al.: 引入了程序依赖图（PDG）的概念，将控制流和数据流统一表示。

## 二、前沿热点（2024-2026 精选）

### 1. AI驱动的编译优化

- **Large Language Models for Compiler Optimization (arXiv 2023/10)**: 首次证明7B参数的Transformer模型可以从零开始学习优化LLVM IR，在代码大小上超越-Oz3.0%。这是Meta AI将LLM引入编译流程的标志性工作。
- **BaCO: A Fast and Portable Bayesian Compiler Optimization Framework (ASPLOS 2023)**: 解决了传统Auto-tuning（自动调优）搜索速度慢的问题，使用贝叶斯优化快速找到最佳Pass序列。比Google的AutoScheduler快几个数量级。
- **The Next 700 Machine Learning-Enabled Compiler Optimizations (CC 2024)**: 综述性展望，讨论了如何用ML替代700种传统启发式优化规则，是领域的方向标。

### 2. MLIR与异构计算编程

- **MLIR: A Compiler Infrastructure for the End of Moore's Law (arXiv 2020)**: Chris Lattner关于MLIR的“宣言”。阐述了多级IR（Multi-Level IR）如何解决领域特定硬件（DSAs）的编译难题。必读。
- **Stardust: Compiling Sparse Tensor Algebra to a Reconfigurable Dataflow Architecture (CGO 2025)**: Fredrik Kjolstad组的工作（稀疏张量编译的权威）。展示了如何将高级张量运算编译到可重构数据流架构（类似CGRA），性能接近手写代码。

### 3. 安全与形式化验证

- **Alive2: Bounded Translation Validation for LLVM (PLDI 2021)**: 工业界事实上的“金标准”。它能自动证明LLVM的优化Pass是否保持了程序语义（Semantics-Preserving），抓住了无数编译器Bug。
- **SmoothE: Differentiable E-Graph Extraction (ASPLOS 2025 Best Paper)**: 将可微编程（Differentiable Programming）引入到E-graph（等价类图）提取中，使得编译器在寻找最优表达式时能利用梯度下降，这是PL和ML的深度交叉。

### 4. 量子编译

- **A Resource-Virtualized and Hardware-Aware Quantum Compilation Framework for Real Quantum Processors (Research 2025)**: 清华团队针对中国超导量子芯片“Quafu”提出的框架。引入了“量子资源虚拟化”概念，像云计算一样管理量子比特，编译保真度显著提升。
- **Quantum Circuit Compilation for Trapped-Ion Processors With the Drive-Through Architecture (IEEE TQE 2025)**: 针对新型“穿越架构”囚禁离子量子处理器，提出了考虑离子距离和动态布局的编译算法，代表了离子阱方向的最新进展。



# 国内编译器未来展望

## ■ 1. 市场长期不饱和。

- 编译器开发有一定的门槛，且短期回报不高。愿意在该领域钻研的人数不多，也导致国内编译方面的人才相对不足，许多编译器开发团队存在招聘困难与人才紧缺情况。因此未来一段时间，编译器领域将仍较有前景。

## ■ 2. 发展方向多元。目前对编译器的需求可分为三类：

- 第一类是编译器优化：各个大厂要维持业务线高效运转，需要做基础功能的保障。
- 第二类是AI编译器：当前通用芯片已难以满足AI需求，已有大量DSA芯片涌现。面临前端框架和各种处理器后端接入费时费力等问题，对AI编译器的需求较大。目前AI编译器正处于快速发展期，大厂相继投入巨大资金和人力以取得先发优势。
- 第三类是新硬件架构工具链：随着国产芯片进程的推进，需要自研上层编译软件及相关工具链的支持。

# 内容线索

√ 什么叫编译程序

√. 编译过程概述

√. 编译程序的结构

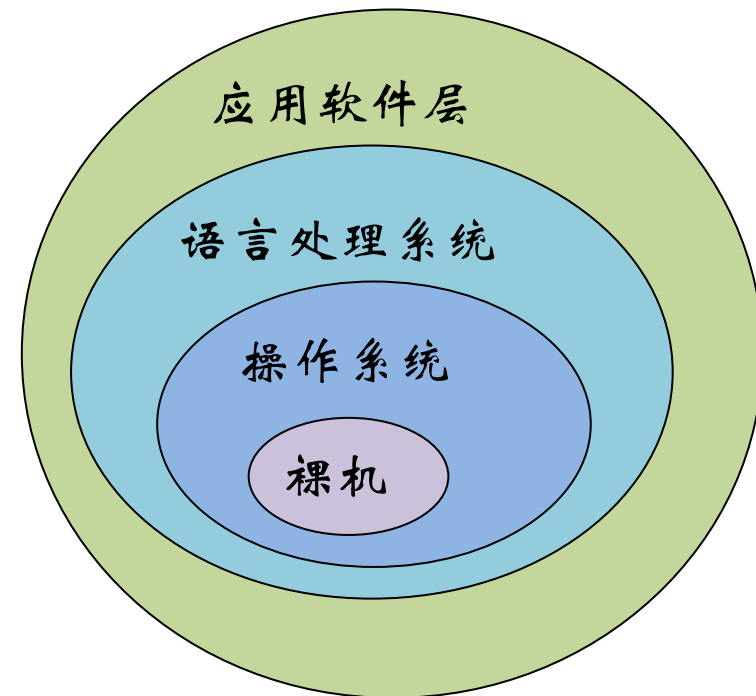
√. 编译程序的生成

5. 总结

# 编译程序在计算机系统中的地位

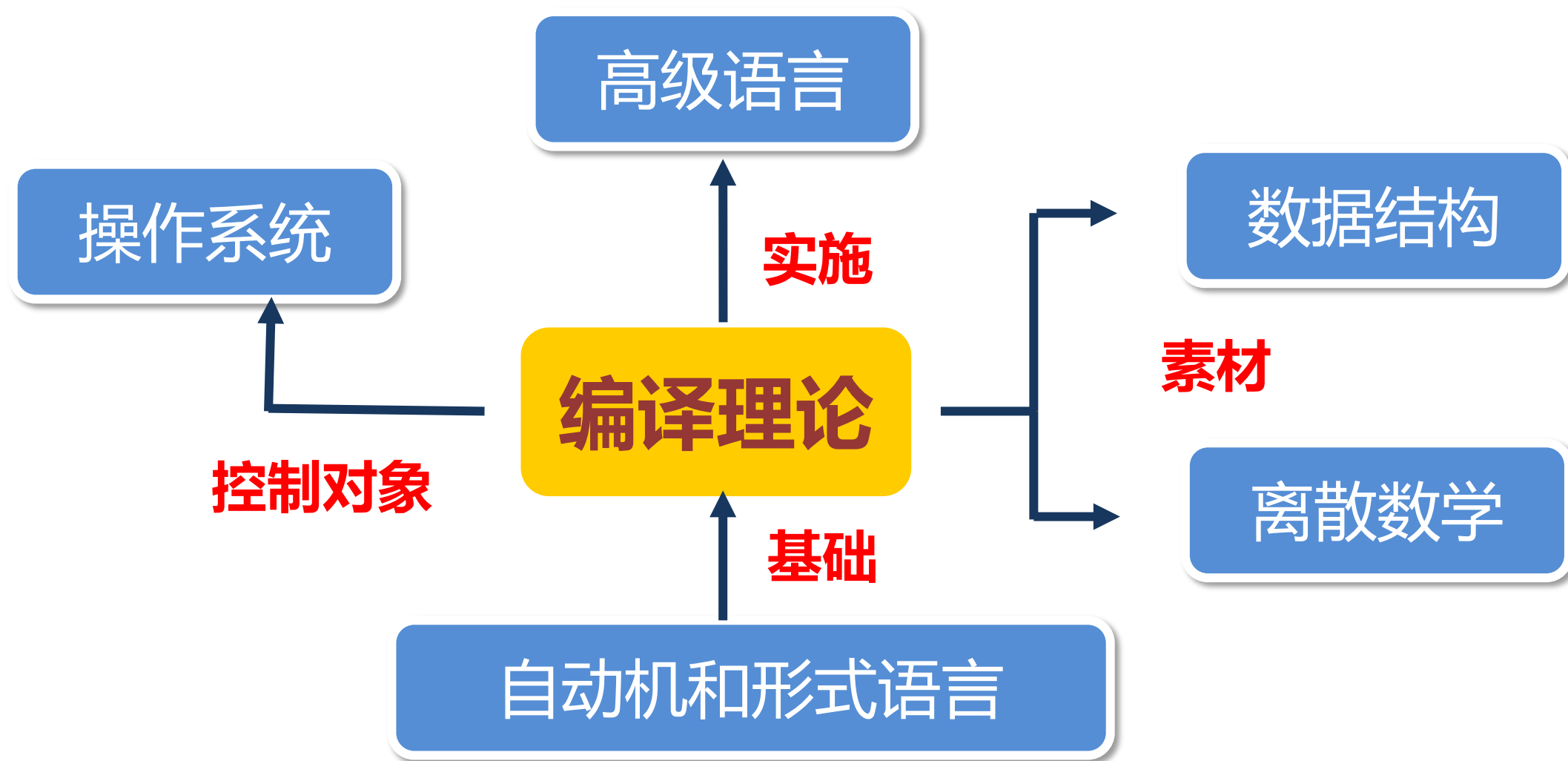
## ■ 编译系统是一种系统软件

- 软件：计算机系统中的程序及其文档。
- 系统软件：居于计算机系统中最靠近硬件的一层，其他软件一般都通过系统软件发挥作用。和具体的应用领域无关，如编译系统和操作系统等。
  - 语言处理系统：把软件语言书写的各种程序处理成可在计算机上执行的程序，如编译系统。



计算机系统

# 编译理论与其他课程的关系



# 为什么要学习编译原理

- **必修主干课程**，操作系统和编译系统构成程序设计与计算机之间的基本界面。
- 通过学习编译原理可以更好地**理解程序语言的内部机制**，从而更好地**理解和运用程序设计语言**。
- 能运用编译程序构造的原理和技术完成相关软件工具的设计和开发工作。

# 教材及参考书

## ■ 教材

- 《程序设计语言编译原理》（第3版），陈火旺等著，国防工业出版社。

## ■ 参考书

- Aho, Lam, Sethi, Ullman, Compilers: Principles, Techniques, and Tools, Addison-Wesley. (龙书)
- 《Modern Compiler Implementation in C》（《现代编译原理C语言描述》）Andrew W.Appel著（赵克佳等译），人民邮电出版社。
- 《编译原理》，吕映芝、张素琴，清华大学出版社。
- 《编译方法》，胡笔蕊等，电子工业出版社。
- .....

# 工程认证要求

| 支撑毕业要求指标点                              | 课程如何支撑毕业要求                                                                 |
|----------------------------------------|----------------------------------------------------------------------------|
| <b>1-4具备对复杂工程问题进行计算机求解的专业知识</b>        | 通过课程知识的学习，使学生深入理解程序语言编译的原理，掌握编译程序设计的基本知识和基本方法，为今后解决复杂的计算机系统设计和实现奠定扎实的专业知识。 |
| <b>2-3具备对复杂工程问题进行分析 and 求解的能力</b>      | 通过课程学习，结合词法分析器设计、语法分析器设计等大作业，培养学生运用有限状态自动机、BNF范式、语法分析算法等解决程序语言编译具体问题的能力。   |
| <b>3-2熟练掌握对复杂计算机系统进行分析 and 总体设计的方法</b> | 通过词法分析器设计、语法分析器设计等大作业，培养学生针对实际设计任务开展方案设计和系统实现的能力。                          |
| <b>10-1具备撰写复杂工程方案技术报告 and 设计文稿的能力</b>  | 通过要求学生撰写课程实验设计报告，培养学生撰写技术报告和方案的能力。                                         |

# 要求及学习方法

**课程特点：理论性强，算法复杂**

## ■ 平时

- 出勤
- 听课+思考
- 认真完成课堂和课后作业

## ■ 课堂讨论

- 积极参与课堂讨论

## ■ 大作业

## ■ 期末: 闭卷笔试



- **出勤及课堂表现：10%**
- **平时作业：10%**
- **大作业1（词法和语法分析）：10%**
- **大作业2（中间代码生成）：10%**
- **期末考试：60%**

# 本书结构

- 第一章 引论
- 第二章 高级语言及其语法描述
- 第三章 词法分析 ★
- 第四章 语法分析——自上而下分析 ★
- 第五章 语法分析——自下而上分析 ★
- 第六章 属性文法和语法制导翻译 ☆
- 第七章 语义分析和中间代码产生 ★
- 第八章 符号表
- 第九章 运行时存储空间组织 ★
- 第十章 优化 ☆
- 第十一章 目标代码生成 ☆

## 作业题

- 高级程序设计语言有哪两种执行方式？其特点是什么？
- 什么是编译程序？它的功能是什么？
- 一个编译程序由哪几个阶段构成？

## 思考题

- 比较你所使用过的一些语言的编译程序：他们的编译速度，出错信息的可读性，有无优化选择等？